Status	Finished
Started	Tuesday, 22 September 2026, 3:44 PM
Completed	Tuesday, 29 September 2026, 4:08 PM
Duration	7 days
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Consider the below words as key words and check the given input is key word or not.

keywords: {break, case, continue, default, defer, else, for, func, goto, if, map, range, return, struct, type, var}

Input format:

Take string as an input from stdin.

Output format:

Print the word is key word or not.

Example Input:

break

Output:

break is a keyword

Example Input:

IF

Output:

IF is not a keyword

For example:

Input	Result
break	break is a keyword
IF	IF is not a keyword

Answer: (penalty regime: 0 %)

```
def check_keyword():
    # Set of given keywords
    keywords = {
        "break", "case", "continue", "default", "defer", "else",
        "for", "func", "goto", "if", "map", "range",
        "return", "struct", "type", "var"
    }

    # Read input from stdin and strip any surrounding whitespace/newlines
    word = input().strip()

    # Check if the word is in the set of keywords (case-sensitive)
    if word in keywords:
        print(f"{word} is a keyword")
    else:
        print(f"{word} is not a keyword")

if __name__ == "__main__":
    check_keyword()
```

	Input	Expected	Got	
✓	break	break is a keyword	break is a keyword	✓
✓	IF	IF is not a keyword	IF is not a keyword	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Find if a String2 is substring of String1. If it is, return the index of the first occurrence. else return -1.

Sample Input 1

thistest123string

123

Sample Output 1

8

Answer: (penalty regime: 0 %)

```
def find_substring_index():
    # Read String1 and String2 from stdin
    string1 = input().strip()
    string2 = input().strip()

    # .find() returns the lowest index of the substring, or -1 if not found
    index = string1.find(string2)

    print(index)

if __name__ == "__main__":
    find_substring_index()
```

	Input	Expected	Got	
✓	thistest123string 123	8	8	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Given a string, determine if it is a palindrome, considering only alphanumeric characters and ignoring cases.

Note: For the purpose of this problem, we define empty string as valid palindrome.

Example 1:

```
Input:
A man, a plan, a canal: Panama

Output:
1
```

Example 2:

```
Input:
race a car

Output:
0
```

Constraints:

- `s` consists only of printable ASCII characters.

Answer: (penalty regime: 0 %)

```python
import sys

def is_palindrome():
    # Read the entire input from stdin (handles multi-word strings and spaces correctly)
    s = sys.stdin.read().strip()

    # Filter out non-alphanumeric characters and convert to lowercase
    filtered = "".join(ch.lower() for ch in s if ch.isalnum())

    # Check if the filtered string equals its reverse
    if filtered == filtered[::-1]:
        print(1)
    else:
        print(0)

if __name__ == "__main__":
    is_palindrome()
```

	Input	Expected	Got	
✓	A man, a plan, a canal: Panama	1	1	✓
✓	race a car	0	0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Given a **non-empty** string s and an abbreviation abbr, return whether the string matches with the given abbreviation.

A string such as "word" contains only the following valid abbreviations:

["word", "1ord", "w1rd", "wo1d", "wor1", "2rd", "w2d", "wo2", "1o1d", "1or1", "w1r1", "1o2", "2r1", "3d", "w3", "4"]

Notice that only the above abbreviations are valid abbreviations of the string "word". Any other string is not a valid abbreviation of "word".

Note:

Assume s contains only lowercase letters and abbr contains only lowercase letters and digits.

Example 1:

Input

internationalization
i12iz4n

Output

true

Explanation

Given **s** = "internationalization", **abbr** = "i12iz4n":

Return true.

Example 2:

Input

apple
a2e

Output

false

Explanation

Given **s** = "apple", **abbr** = "a2e":

Return false.

Answer: (penalty regime: 0 %)

```python
import sys

def valid_word_abbreviation(s: str, abbr: str) -> bool:
    i, j = 0, 0
    m, n = len(s), len(abbr)

    while i < m and j < n:
        # If the character in abbreviation is a digit
        if abbr[j].isdigit():
            # Leading zeros are invalid (e.g., "0" or "01")
            if abbr[j] == '0':
                return False

            # Parse the full number
            num = 0
            while j < n and abbr[j].isdigit():
                num = num * 10 + int(abbr[j])
                j += 1

            # Advance the pointer in the main string by the parsed number
            i += num
        else:
            # If it's a letter, characters must match
            if s[i] != abbr[j]:
                return False
            i += 1
            j += 1

    # Both pointers must reach the end of their respective strings
    return i == m and j == n

def main():
    # Read all lines from standard input
    lines = sys.stdin.read().splitlines()
    if len(lines) >= 2:
        s = lines[0].strip()
        abbr = lines[1].strip()

        result = valid_word_abbreviation(s, abbr)
        # Print lowercase 'true' or 'false' as shown in the examples
        print(str(result).lower())

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	internationalization i12iz4n	true	true	✓
✓	apple a2e	false	false	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Write a Python program to get one string and reverses a string. The input string is given as an array of characters `char[]`.

You may assume all the characters consist of printable ascii characters.

Example 1:

```
Input:
hello
Output:
olleh
```

Example 2:

```
Input:
Hannah
Output:
hannaH
```

Answer: (penalty regime: 0 %)

```
import sys

def reverse_string(s):
    """
    Do not return anything, modify s in-place instead.
    """
    left, right = 0, len(s) - 1
    while left < right:
        # Swap characters using two pointers
        s[left], s[right] = s[right], s[left]
        left, right = left + 1, right - 1

def main():
    # Read the input string from standard input
    line = sys.stdin.read().strip()
    if line:
        # Convert string to a list of characters (char[])
        char_array = list(line)

        # Reverse the array in-place
        reverse_string(char_array)

        # Join back into a string for printing
        print("".join(char_array))

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	hello	olleh	olleh	✓
✓	Hannah	hannaH	hannaH	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Assume that the given string has enough memory.

Don't use any extra space(IN-PLACE)

Sample Input 1

a2b4c6

Sample Output 1

aabbbbccccccc

Answer: (penalty regime: 0 %)

```
import sys

def expand_string_in_place(char_list):
    """
    Expands a string like 'a2b4c6' into 'aabbbbccccccc' in-place
    by working backwards from the end of the array.
    """
    n = len(char_list)

    # Step 1: First, calculate the final expanded length by scanning the original string
    original_length = 0
    i = 0
    while i < n and char_list[i] != '\0':
        # If it's a character, look ahead for its number
        if not char_list[i].isdigit():
            j = i + 1
            num = 0
            while j < n and char_list[j].isdigit():
                num = num * 10 + int(char_list[j])
                j += 1
            original_length += num
            i = j
        else:
            i += 1

    # In Python, we can extend the list with null/blank padding to simulate pre-allocated "enough memory"
    # Let's find the exact length of the compressed string first (ignoring trailing whitespace/nulls)
    s = "".join(char_list).strip()

    # Calculate target length
    target_len = 0
    i = 0
    while i < len(s):
        ch = s[i]
        i += 1
        num = 0
        while i < len(s) and s[i].isdigit():
            num = num * 10 + int(s[i])
            i += 1
        target_len += num

    # Resize/extend the character list with spaces to act as the "extra memory" at the end
    char_list.extend([' '] * (target_len - len(char_list)))

    # Step 2: Expand from right to left (backwards) to avoid overwriting unread data
    read_ptr = len(s) - 1
    write_ptr = target_len - 1

    while read_ptr >= 0:
        # Read the count (digits)
        num = 0
        base = 1
```

	Input	Expected	Got	
✓	a2b4c6	aabbbbcccccc	aabbbbcccccc	✓
✓	a12b3d4	aaaaaaaaaaaabbbdddd	aaaaaaaaaaaabbbdddd	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Given a string S which is of the format USERNAME@DOMAIN.EXTENSION, the program must print the EXTENSION, DOMAIN, USERNAME in the reverse order.

Input Format:

The first line contains S.

Output Format:

The first line contains EXTENSION.
The second line contains DOMAIN.
The third line contains USERNAME.

Boundary Condition:

1 <= Length of S <= 100

Example Input/Output 1:

Input:

abcd@gmail.com

Output:

com
gmail
abcd

For example:

Input	Result
arvijayakumar@rajalakshmi.edu.in	edu.in rajalakshmi arvijayakumar

Answer: (penalty regime: 0 %)

```
import sys

def main():
    # Read all lines from standard input to handle multiple test cases
    lines = sys.stdin.read().splitlines()

    for line in lines:
        s = line.strip()
        if not s:
            continue

        # Split username from domain.extension
        username, rest = s.split('@', 1)

        # Split domain and extension at the first dot
        domain, extension = rest.split('.', 1)

        # Print in the required reverse order
        print(extension)
        print(domain)
        print(username)

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	abcd@gmail.com	com gmail abcd	com gmail abcd	✓
✓	arvijayakumar@rajalakshmi.edu.in	edu.in rajalakshmi arvijayakumar	edu.in rajalakshmi arvijayakumar	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

A pangram is a sentence where every letter of the English alphabet appears at least once.

Given a string sentence containing only lowercase English letters, return true if sentence is a pangram, or false otherwise.

Example 1:

Input:

thequickbrownfoxjumpsoverthelazydog

Output:

true

Explanation: sentence contains at least one of every letter of the English alphabet.

Example 2:

Input:

arvijayakumar

Output: false

Constraints:

1 <= sentence.length <= 1000

sentence consists of lowercase English letters.

For example:

Test	Result
`print(checkPangram('thequickbrownfoxjumpsoverthelazydog'))`	`true`
`print(checkPangram('arvijayakumar'))`	`false`

Answer: (penalty regime: 0 %)

Reset answer

```
def checkPangram(sentence: str) -> str:
    # Check if unique characters equal 26 and return lowercase string
    if len(set(sentence)) == 26:
        return "true"
    return "false"
```

	Test	Expected	Got	
✓	`print(checkPangram('thequickbrownfoxjumpsoverthelazydog'))`	`true`	`true`	✓

	Test	Expected	Got	
✓	`print(checkPangram('arvijayakumar'))`	`false`	`false`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

The program must accept **N** series of keystrokes as string values as the input. The character **^** represents undo action to clear the last entered keystroke. The program must print the string typed after applying the undo operations as the output. If there are no characters in the string then print **-1** as the output.

Boundary Condition(s):

1 <= N <= 100
1 <= Length of each string <= 100

Input Format:

The first line contains the integer N.
The next N lines contain a string on each line.

Output Format:

The first N lines contain the string after applying the undo operations.

Example Input/Output 1:

Input:

3
Hey ^ goooo^^glee^
lucke^y ^charr^ms
ora^^nge^^^^

Output:

Hey google
luckycharms
-1

Answer: (penalty regime: 0 %)

```
import sys

def process_keystrokes():
    # Read all lines from standard input
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    # The first line contains N
    n = int(input_data[0].strip())
    results = []

    # Process the next N lines
    for i in range(1, n + 1):
        if i < len(input_data):
            s = input_data[i]
        else:
            s = ""

        stack = []
        for char in s:
            if char == '^':
```

```
                # Undo: remove the last entered character if stack is not empty
                if stack:
                    stack.pop()
            else:
                # Type character: push to stack
                stack.append(char)

        # If the resulting string is empty, output -1, else join the stack
        if not stack:
            results.append("-1")
        else:
            results.append("".join(stack))

    # Print all results
    for res in results:
        print(res)

if __name__ == "__main__":
    process_keystrokes()
```

	Input	Expected	Got	
✓	3 Hey ^ goooo^^glee^ lucke^y ^charr^ms ora^^nge^^^^	Hey google luckycharms -1	Hey google luckycharms -1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Given a string s containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:

Open brackets must be closed by the same type of brackets.

Open brackets must be closed in the correct order.

Constraints:

1 <= s.length <= 10^4

s consists of parentheses only '()[]{}'.

For example:

Test	Result
`print(ValidParenthesis("()"))`	`true`
`print(ValidParenthesis("()[]{}"))`	`true`
`print(ValidParenthesis("(]"))`	`false`

Answer: (penalty regime: 0 %)

Reset answer

```
def ValidParenthesis(s: str) -> str:
    stack = []
    mapping = {")": "(", "}": "{", "]": "["}

    for char in s:
        if char in mapping:
            top_element = stack.pop() if stack else '#'
            if mapping[char] != top_element:
                return "false"
        else:
            stack.append(char)

    return "true" if len(stack) == 0 else "false"
```

	Test	Expected	Got	
✓	`print(ValidParenthesis("()"))`	`true`	`true`	✓
✓	`print(ValidParenthesis("()[]{}"))`	`true`	`true`	✓
✓	`print(ValidParenthesis("(]"))`	`false`	`false`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.